HPC Project Report

Leonardo Antonio Riola, Davide Donati, Giacomo Girotto

1 Assignment 1

1.1 Introduction and objectives

The goal of this assignment is to implement a simulator for the propagation of a damped wave, governed by the following partial differential equation:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u \tag{1}$$

where $u(x, y, t)$ is the wave height on a two-dimensional plane over time, γ is the damping coefficient, and c is the propagation speed. The simulator computes u at every point (x, y) as t advances, with the final goal of producing an MP4 video of the simulation.

1.2 Implementation

The continuous model is discretised on a regular grid: space is divided into points spaced by Δx and Δy, and time into steps of Δt. Each grid point is identified by indices (i, j) and each instant by an index t, so that $u(x, y, t)$ becomes a matrix of values $u^t_{i,j}$. The derivatives in the equation are approximated with **central differences method**.

The temporal evolution is handled with three $M \times M$ matrices, `u_past`, `u` and `u_fut`, dynamically allocated with `malloc` and representing the wave state at times $t-1$, t and $t+1$ respectively. All elements are initialised to zero except for a single impulse at the centre of the grid $(M/2, M/2)$, used to start the perturbation. The impulse amplitude, propagation speed and damping coefficient used throughout this work are:

Impulse	**Propagation speed c**	**Damping coefficient γ**
-49	0.33	0.187

The core of this assignment is the parallelization of the update loop with **OpenMP**. A `#pragma omp parallel` region is opened once before the time loop, and the two nested spatial loops are distributed across threads with `#pragma omp for schedule(static) collapse(2)`: `schedule(static)` splits the iterations into equal-sized chunks assigned before execution, while `collapse(2)` fuses the i and j loops into a single $M \times M$ iteration space, giving the scheduler far more, finer-grained work units to balance across cores. Each thread thus computes $u^{t+1}_{i,j}$ for a disjoint set of coordinates, independently and concurrently with the others.

Once `u_fut` has been fully computed for the current timestep, it must be written to a PGM file; this is done inside a `#pragma omp single` block, so that only one thread performs the write while the rest wait at the implicit barrier. To avoid the cost of deep-copying entire matrices between timesteps, the temporal advance is implemented as a three-way pointer rotation (`u_past` takes the address of `u`, `u` takes that of `u_fut`, and `u_fut` is recycled for the next iteration), performed by the same thread that just wrote the file. This keeps the pointer swap race-free and avoids any extra synchronization beyond the barrier already required for the write.

1.2.1 Identified inefficiencies and corrective transformations

All profiling and testing in this section was performed using the *Edu_skylake* partition, 32 threads and a workload size of $M = 1000$. The aim was to identify the main bottlenecks and improve the source code.

We profiled the initial implementation with **Intel VTune Profiler**. *Hotspots* analysis identifies which functions in an application consume the largest share of execution time, using low-overhead sampling to build a ranked list of hotspot functions as a starting point for optimization [1]. *Threading* analysis instead evaluates how efficiently the application uses the available processor cores, exposing inefficiencies in threading-runtime usage and contention at synchronization points that leave threads idle and prevent full processor utilization, summarized through its *Effective CPU Utilization* metric [2]. Combining the two analyses lets us separate genuine computation time from time lost to idle spinning at synchronization points.

Hotspot analysis showed that `gomp_team_barrier_wait_end` and `fwrite` accounted for 79.4% and 9.7% of total CPU time respectively, against only 2.2% for the matrix computation itself [1]: the bottleneck was therefore not the stencil, but excessive synchronization and I/O overhead. Three successive transformations addressed this, each verified either via VTune or direct code inspection.

Improvement 1: buffered frame output. Frames were originally written pixel by pixel with `fwrite()` inside a single-threaded `#pragma omp single` block. We replaced this with an in-memory buffer, filled in parallel via a dedicated `#pragma omp for` and flushed to disk with one `fwrite()` call, cutting I/O calls from one per pixel to one per frame. This did introduce a new barrier, however, since the buffer-fill loop is a separate worksharing construct from the stencil loop.

Improvement 2: loop fusion. Since the stencil and the buffer-fill iterate over the exact same (i, j) range, we merged them into a single `#pragma omp for`, writing each scaled pixel immediately after computing `u_fut[i][j]`. This removes the extra barrier introduced above, with no loss of parallel work.

Improvement 3: parallel initialization and scheduling policy. The matrix initialization, originally performed sequentially by a single thread, was parallelized with `#pragma omp parallel for`. By Amdahl's law, this serial fraction would otherwise cap the achievable speedup regardless of how well the main loop scales. On a multi-socket compute node, each CPU socket has its own directly-attached memory bank: cores access memory local to their own socket faster than memory attached to another socket, a design known as NUMA (Non-Uniform Memory Access). Under the default *first-touch* policy, a physical page is assigned to the socket of whichever thread writes to it first, we therefore gave the initialization loop the same schedule as the stencil loop, so that each thread's first-touch region matches the region it repeatedly accesses afterwards. Profiling the complete pipeline (including I/O) showed, however, that `schedule(guided)` outperforms `schedule(static)` in practice [**?**]: dynamic work redistribution better compensates for the load imbalance introduced by the single-threaded frame-writing step, so `guided` was retained for the final implementation. As shown later in the testing section, isolating the computation from I/O reverses this result, confirming that NUMA locality does matter once the synchronization overhead of I/O is removed.

After these three improvements, VTune Hotspot analysis showed the CPU time spent in `gomp_team_barrier_wait_end` dropping to 57.5%, while time spent in the matrix computation itself rose to 15.7%; I/O (`fwrite`) time fell below 0.3% [2]. The Threading analysis, however, revealed that the program was still using only 23% of the available CPUs, for an average effective utilization of 7.362 out of 32 threads: a clear sign that, at $M = 1000$, the workload was simply too small for the number of threads requested. This motivated the next round of tests, which vary the workload size and other parameters to see how resource utilization changes.

[1] gjgjgj

[2] See "output_1920920.txt"

1.3 Testing and Results

1.3.1 Sweep of workload size

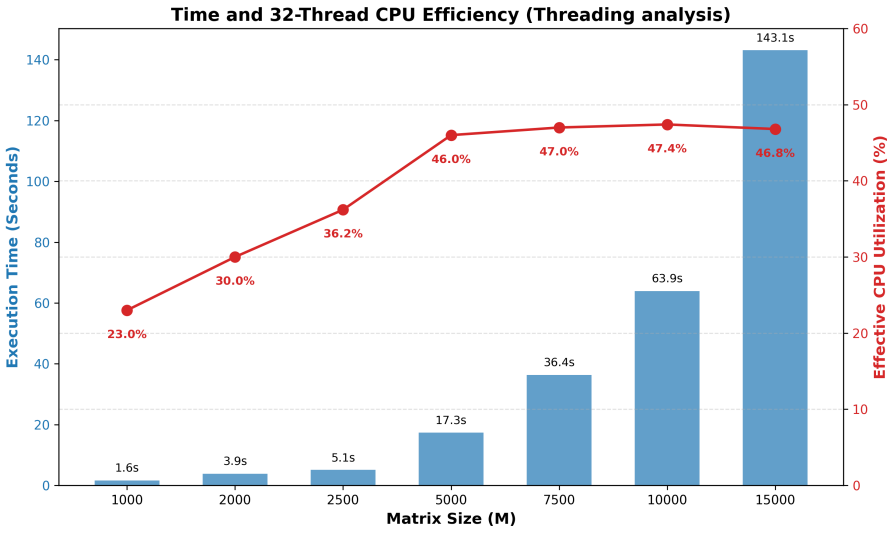

Figure 1: Execution time and true CPU efficiency scaling with varying grid sizes (M), using 32 threads.

As shown in Figure 1, effective CPU utilization rises steeply as the workload grows from $M = 1000$ to $M = 5000$, from 23.0% to 46.0%, then continues to increase more slowly, peaking at 47.4% for $M = 10000$ [3]. This indicates that the program alone cannot make full use of the available resources, mainly because of the serialized I/O; a larger computational workload partially hides this cost, but it remains an unavoidable bottleneck.

Execution time scales quadratically ($O(M^2)$) with the grid size, as expected for the spatial computation: going from $M = 5000$ to $M = 15000$ (a 3x increase in linear size) increases execution time by 8.25×, close to the ideal 9× predicted by quadratic growth – the small deviation is attributable to constant thread-management overhead.

1.3.2 Sweep of number of threads at $M = 15000$

The remaining tests were run on the `Edu_sapphire` partition (2 nodes, 48 physical and 96 logical cores each); each node is dual-socket, with 24 physical cores per socket.

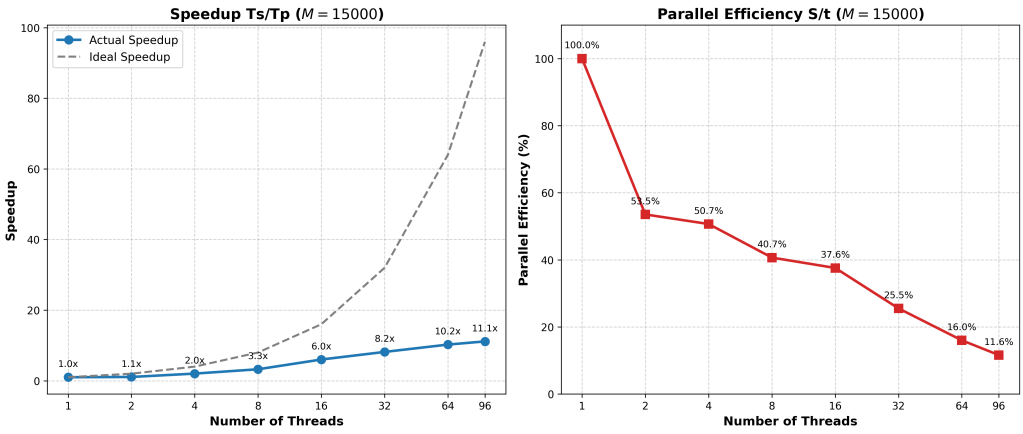

Figure 2: Speedup and parallel efficiency with varying number of threads.

[3] See output_1920920.txt, output_1920886.txt, output_1920889.txt, output_1920892.txt, output_1920894.txt, output_1920842.txt, output_1920899.txt

Figure 2 shows two distinct regimes. For $P \leq 32$, all threads run on independent physical cores; the residual efficiency loss is explained by the OpenMP barrier cost identified with VTune, which grows with team size. For $P = 64$ and $P = 96$, thread count exceeds the number of physical cores (48), so multiple logical threads begin sharing the execution units, caches and memory bandwidth of the same physical core through Hyper-Threading: each core exposes two logical processors to the OS, which share the same execution units, caches and memory bandwidth [3].

We repeated the same test with the I/O operations disabled, so that the program only computes the matrix updates. [4]

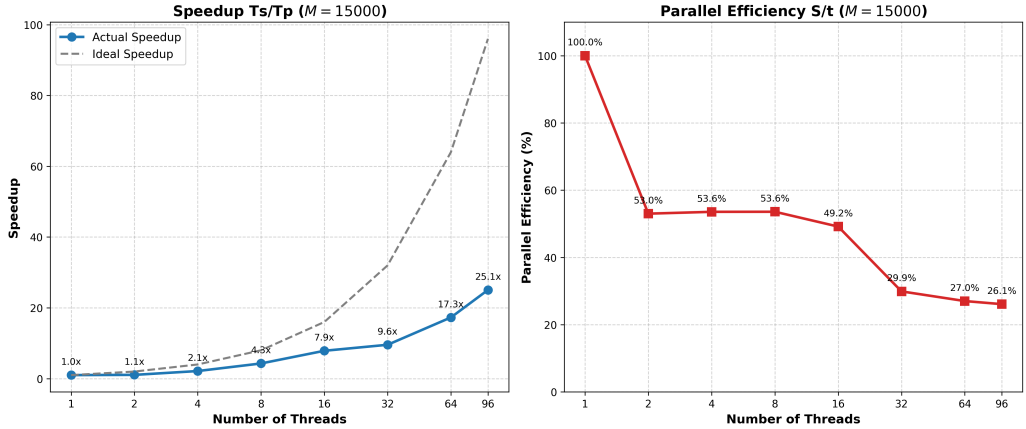

Figure 3: Speedup and parallel efficiency with varying number of threads, I/O disabled.

As Figure 3 shows, both metrics improve significantly: at $P = 96$ speedup more than doubles, from 11.1× to 25.1×, and parallel efficiency grows from 11.6% to 26.1%. This confirms that a substantial share of the overhead seen in the full version came from the I/O operations and their associated synchronization barriers, rather than from the stencil computation itself.

Finally, we replaced `schedule(guided)` with `schedule(static)` in both the initialization and the stencil loop, to restore the NUMA alignment mentioned earlier. [5] Once thread count exceeds 24, work is necessarily split across both sockets, and accesses to memory owned by the other socket incur the added latency of the inter-socket interconnect.

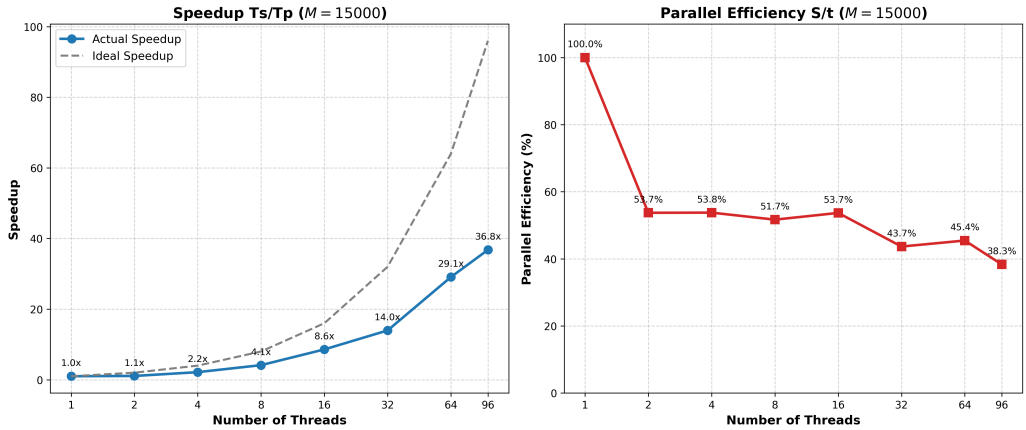

Figure 4: Speedup and parallel efficiency with varying number of threads, I/O disabled, `schedule(static)`.

As shown in Figure 4, the static schedule substantially mitigates this effect: the efficiency drop

[4] See output_1925400.txt, output_1925398.txt, output_1925396.txt, output_1925394.txt, output_1925399.txt, output_1925397.txt, output_1925395.txt, output_1925393.txt in folder "results_noio"

[5] See output_1928239.txt, output_1928222.txt, output_1928220.txt, output_1927019.txt, output_1927027.txt, output_1927036.txt in folder "results_noio_static"

between $P = 16$ and $P = 32$ is roughly halved (from -19.3 to -10.0 percentage points relative to `guided`), and efficiency remains markedly higher at $P = 32$ (43.7% vs. 29.9%), $P = 64$ (45.4% vs. 27.0%, a 68% relative gain) and $P = 96$ (38.3% vs. 26.1%), with speedup at $P = 96$ rising from $25.1\times$ to $36.8\times$. This is consistent with the static schedule preserving the same thread-to-row mapping established during initialization, which keeps most memory accesses local to each thread's own socket even as the workload spans both NUMA domains.

2 Assignment 2

2.1 Introduction

Following the initial wave simulator developed with OpenMP, this second assignment extends the application to execute multiple concurrent simulations on an HPC system using a hybrid MPI and OpenMP programming model. The goal is to study multi-wave interaction and interference dynamics under different physical conditions. While OpenMP continues to handle data parallelism by accelerating the 2D stencil computations across local CPU cores, MPI is introduced to manage task parallelism. Specifically, the system executes three distinct simulation scenarios simultaneously within a single execution run:

- Simulation 1: Reproduces the baseline single-wave scenario initiated by a single central impulse at $t = 0$.
- Simulation 2: Models wave superposition and interference phenomena by initiating two simultaneous impulses at different spatial locations at $t = 0$.
- Simulation 3: Introduces time-delayed interference dynamics by generating a second impulse at a spatial location after a predetermined time-shift ($t = t_{start}$).By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

2.2 Implementation

Building upon the code developed in Assignment 1, the implementation extends the simulator to handle multiple concurrent runs. The core numerical scheme for the time-stepping propagation and the mapping function converting field values into grayscale images (**save_pgm**) remain identical to the first version, ensuring consistency in wave physics and visualization. The execution starts by initializing the MPI environment with **MPI_Init**, retrieving the total number of processes with **MPI_Comm_size**, and identifying the rank ID of each process with **MPI_Comm_rank**. To run the three required simulations simultaneously, the program verifies that at least 3 processes are allocated. Task parallelism is managed through conditional branching based on the process rank, where each rank executes a specific scenario and saves its frames into a dedicated directory:

- **Rank 0** (*sim1*): Handles the baseline single-wave simulation initiated by a central impulse of **-84.0f** at coordinates $(M/2, M/2) = (200, 200)$ at $t = 0$.
- **Rank 1** (*sim2*): Manages the simultaneous two-wave interference scenario, applying the central impulse **-84.0f** at $(200, 200)$ alongside a second impulse of **36.0f** at coordinates $(3M/4, M/3) = (300, 133)$ at $t = 0$.
- **Rank 2** (*sim3*): Manages the time-delayed scenario, applying the central impulse **-84.0f** at $(200, 200)$ at $t = 0$ and scheduling a second impulse of **60.0f** at coordinates $(3M/4, 2M/3) = (300, 266)$ to be injected later at time step $n_{\text{start}} = N/7$.

Each MPI process independently allocates memory using **malloc** for its local grid arrays (**u_prev**, **u_curr**, and **u_next**). While data parallelism within each process is still handled by OpenMP as established in Assignment 1, the time-stepping loop incorporates rank-specific logic. Specifically, the check for **rank == 2** at **n == n_start** is placed inside a **pragma omp single** directive within the time loop. This ensures that the time-delayed second impulse for Simulation 3 is injected into the grid at the exact required time step by only one thread per process, avoiding race conditions and preventing multiple threads from applying the impulse redundantly. The same **pragma omp single** block also safely manages frame saving with **save_pgm** and array pointer swapping without thread interference.

The job execution on the HPC cluster is controlled via a SLURM batch script. The configuration requests 3 MPI tasks (*ntasks=3*) to match the 3 simulation ranks, and reserves 4 CPU cores per task (*cpus-per-task=4*). By setting **export OMP_NUM_THREADS** dynamically to the allocated CPUs per task, each MPI process spawns four OpenMP threads, effectively utilizing a total of 12 CPU cores in parallel across the compute node.

2.3 Test and optimizaion

3 Assignment 3

References

[1] Intel Corporation. Hotspots analysis for CPU usage issues – intel vtune profiler user guide. https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/basic-hotspots-analysis.html.

[2] Intel Corporation. Threading analysis – intel vtune profiler user guide. https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/threading-analysis.html.

[3] Deborah T. Marr, Frank Binns, David L. Hill, Glenn Hinton, David A. Koufaty, J. Alan Miller, and Michael Upton. Hyper-threading technology architecture and microarchitecture. *Intel Technology Journal*, 2002.